

ALU 实验功能改进报告

1 实验原理

1.1 ALU 基本原理

算术逻辑单元 (ALU) 是中央处理器的核心组成部分，负责执行所有的算术运算和逻辑运算。本实验中的原始 ALU 设计支持 12 种基本操作，包括加法、减法、比较、移位和逻辑运算等。

1.2 编码方式改进原理

原始设计采用 12 位独热码作为控制信号，每个操作对应一位控制信号。这种方式的优点是解码简单，但缺点是控制信号位数过多，浪费硬件资源。

4 位二进制编码可以表示 16 种不同的状态，完全可以覆盖原有的 12 种操作，并且还有 4 个冗余状态可以用于扩展新功能。本实验将控制信号从 12 位减少到 4 位，通过组合逻辑解码产生各个操作的使能信号。

1.3 新增操作

1. 有符号比较大于置位
2. 按位同或
3. 低位加载

2 实验内容与修改

2.1 操作码编码表设计

保留原有 12 种操作的功能不变，重新分配 4 位二进制操作码，并新增三种操作，编码表如下：

表 1: 改进后的 ALU 操作码编码表

4 位二进制码	AC22(bit3)	AC23(bit2)	AB6(bit1)	W6(bit0)	ALU 操作
0000	0	0	0	0	无操作
0001	0	0	0	1	加法
0010	0	0	1	0	减法
0011	0	0	1	1	有符号比较, 小于置位
0100	0	1	0	0	无符号比较, 小于置位
0101	0	1	0	1	按位与
0110	0	1	1	0	按位或非
0111	0	1	1	1	按位或
1000	1	0	0	0	按位异或
1001	1	0	0	1	逻辑左移
1010	1	0	1	0	逻辑右移
1011	1	0	1	1	算术右移
1100	1	1	0	0	高位加载
1101	1	1	0	1	有符号比较, 大于置位 (新增)
1110	1	1	1	0	按位同或 (新增)
1111	1	1	1	1	低位加载 (新增)

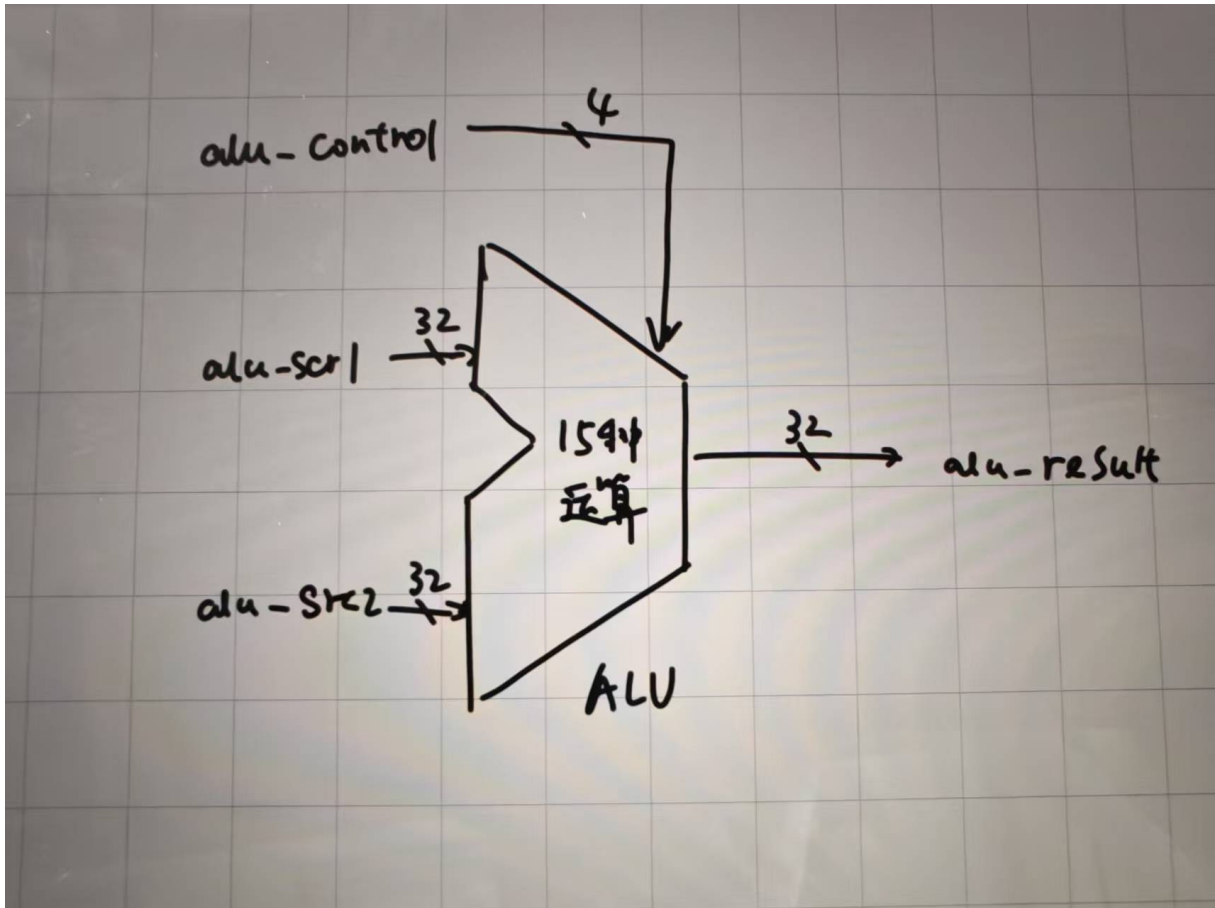


图 1: 原理图

2.2 alu.v 模块修改

这是本次实验的核心修改文件，主要修改内容如下：

1. 模块端口修改：将 12 位独热码控制信号改为 4 位二进制控制信号

```

1 // 修改前
2 module alu(
3     input  [11:0] alu_control, // ALU控制信号
4     input  [31:0] alu_src1,    // ALU操作数1,为补码
5     input  [31:0] alu_src2,    // ALU操作数2,为补码
6     output [31:0] alu_result   // ALU结果
7 );
8
9 // 修改后
10 module alu(
11     input  [3:0] alu_control, // ALU控制信号(4位二进制)
12     input  [31:0] alu_src1,   // ALU操作数1,为补码
13     input  [31:0] alu_src2,   // ALU操作数2,为补码

```

```

14     output [31:0] alu_result    // ALU 结果
15 );

```

2. 控制信号解码修改：移除独热码解码，添加新操作信号并实现 4 位二进制解码

```

1 // 新增三个操作信号
2 wire alu_sgt;    //有符号比较，大于置位(新增)
3 wire alu_xnor;   //按位同或(新增)
4 wire alu_lli;    //低位加载(新增)
5
6 // 4位二进制控制信号解码
7 assign alu_add  = (alu_control == 4'b0001);
8 assign alu_sub  = (alu_control == 4'b0010);
9 assign alu_slt  = (alu_control == 4'b0011);
10 assign alu_sltu = (alu_control == 4'b0100);
11 assign alu_and  = (alu_control == 4'b0101);
12 assign alu_nor  = (alu_control == 4'b0110);
13 assign alu_or   = (alu_control == 4'b0111);
14 assign alu_xor  = (alu_control == 4'b1000);
15 assign alu_sll  = (alu_control == 4'b1001);
16 assign alu_srl  = (alu_control == 4'b1010);
17 assign alu_sra  = (alu_control == 4'b1011);
18 assign alu_lui  = (alu_control == 4'b1100);
19 assign alu_sgt  = (alu_control == 4'b1101); // 新增
20 assign alu_xnor = (alu_control == 4'b1110); // 新增
21 assign alu_lli  = (alu_control == 4'b1111); // 新增

```

3. 新增操作实现：添加三个新操作的结果计算逻辑

```

1 // 按位同或结果为异或结果按位取反
2 assign xnor_result = ~xor_result;
3 // 低位加载：立即数放在低半字节
4 assign lli_result = {16'd0, alu_src2[15:0]};
5
6 // 有符号大于置位结果
7 assign sgt_result[31:1] = 31'd0;
8 assign sgt_result[0]    = (~alu_src1[31] & alu_src2[31]) |
9                          (~(alu_src1[31]^alu_src2[31]) & ~
                              adder_result[31]);

```

4. 结果选择逻辑修改

```

1 // 选择相应结果输出

```



```

2  assign alu_result =
3      (alu_control == 4'b0001) ? add_sub_result :
4      (alu_control == 4'b0010) ? add_sub_result :
5      (alu_control == 4'b0011) ? slt_result :
6      (alu_control == 4'b0100) ? sltu_result :
7      (alu_control == 4'b0101) ? and_result :
8      (alu_control == 4'b0110) ? nor_result :
9      (alu_control == 4'b0111) ? or_result :
10     (alu_control == 4'b1000) ? xor_result :
11     (alu_control == 4'b1001) ? sll_result :
12     (alu_control == 4'b1010) ? srl_result :
13     (alu_control == 4'b1011) ? sra_result :
14     (alu_control == 4'b1100) ? lui_result :
15     (alu_control == 4'b1101) ? sgt_result :
16     (alu_control == 4'b1110) ? xnor_result :
17     (alu_control == 4'b1111) ? lli_result :
18     32'd0;

```

2.3 alu_display.v 模块修改

1. 添加拨码开关输入端口：添加四个拨码开关作为 ALU 控制信号输入

```

1  // 新增：4位拨码开关作为ALU控制信号(从高到低)
2  input alu_ctrl3, // AC22 - 最高位
3  input alu_ctrl2, // AC23
4  input alu_ctrl1, // AB6
5  input alu_ctrl0, // W6 - 最低位

```

2. 修改控制信号输入逻辑：优先使用拨码开关输入，触摸屏输入作为备用

```

1  // ALU控制信号：优先使用拨码开关输入，触摸屏输入作为备用
2  always @(posedge clk)
3  begin
4      if (!resetn)
5      begin
6          alu_control <= 4'd0;
7      end
8      else if (input_valid && input_sel==2'b00)
9      begin
10         alu_control <= input_value[3:0]; // 触摸屏输入控制信号
11     end

```

```

12     else
13     begin
14         alu_control <= {alu_ctrl3, alu_ctrl2, alu_ctrl1, alu_ctrl0};
15         // 拨码开关输入
16     end
17 end

```

3. 修改控制信号显示：调整显示位宽以正确显示 4 位控制信号

```

1 6'd3 :
2 begin
3     display_valid <= 1'b1;
4     display_name  <= "CONTR";
5     display_value <={28'd0, alu_control}; // 显示4位控制信号
6 end

```

2.4 约束文件修改

在约束文件中添加四个拨码开关的引脚约束：

```

1 # 新增：ALU控制信号拨码开关(从高到低)
2 set_property PACKAGE_PIN AC22 [get_ports alu_ctrl3]
3 set_property PACKAGE_PIN AC23 [get_ports alu_ctrl2]
4 set_property PACKAGE_PIN AB6  [get_ports alu_ctrl1]
5 set_property PACKAGE_PIN W6   [get_ports alu_ctrl0]
6
7 set_property IOSTANDARD LVCMOS33 [get_ports alu_ctrl3]
8 set_property IOSTANDARD LVCMOS33 [get_ports alu_ctrl2]
9 set_property IOSTANDARD LVCMOS33 [get_ports alu_ctrl1]
10 set_property IOSTANDARD LVCMOS33 [get_ports alu_ctrl0]

```

3 上箱验证

3.1 硬件连接

按照约束文件的设置，将四个拨码开关 AC22、AC23、AB6、W6 分别连接到 ALU 控制信号的最高位到最低位。

3.2 验证步骤

1. 将编译好的比特流文件下载到 FPGA 开发板
2. 通过拨码开关 input_sel 选择输入源操作数 1 和源操作数 2，使用触摸屏输入具体数值
3. 通过四个控制拨码开关设置不同的操作码
4. 观察触摸屏上显示的运算结果是否正确

3.3 验证结果

上箱验证结果与仿真结果完全一致：

1. 所有原有操作均能正确执行
2. 新增的三种操作功能正常
3. 拨码开关控制灵敏，操作便捷
4. 触摸屏显示清晰，输入输出正常

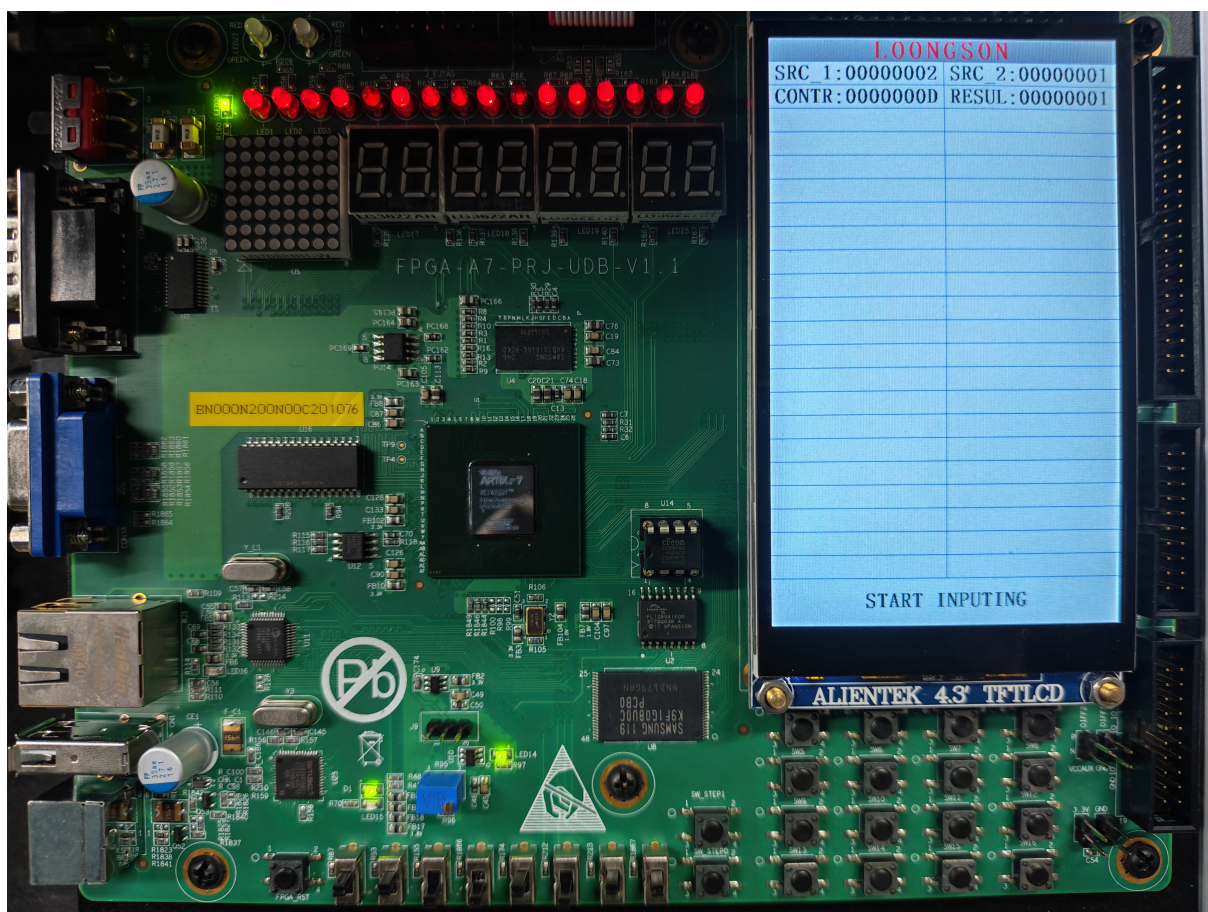


图 2: 有符号比较大于置位功能

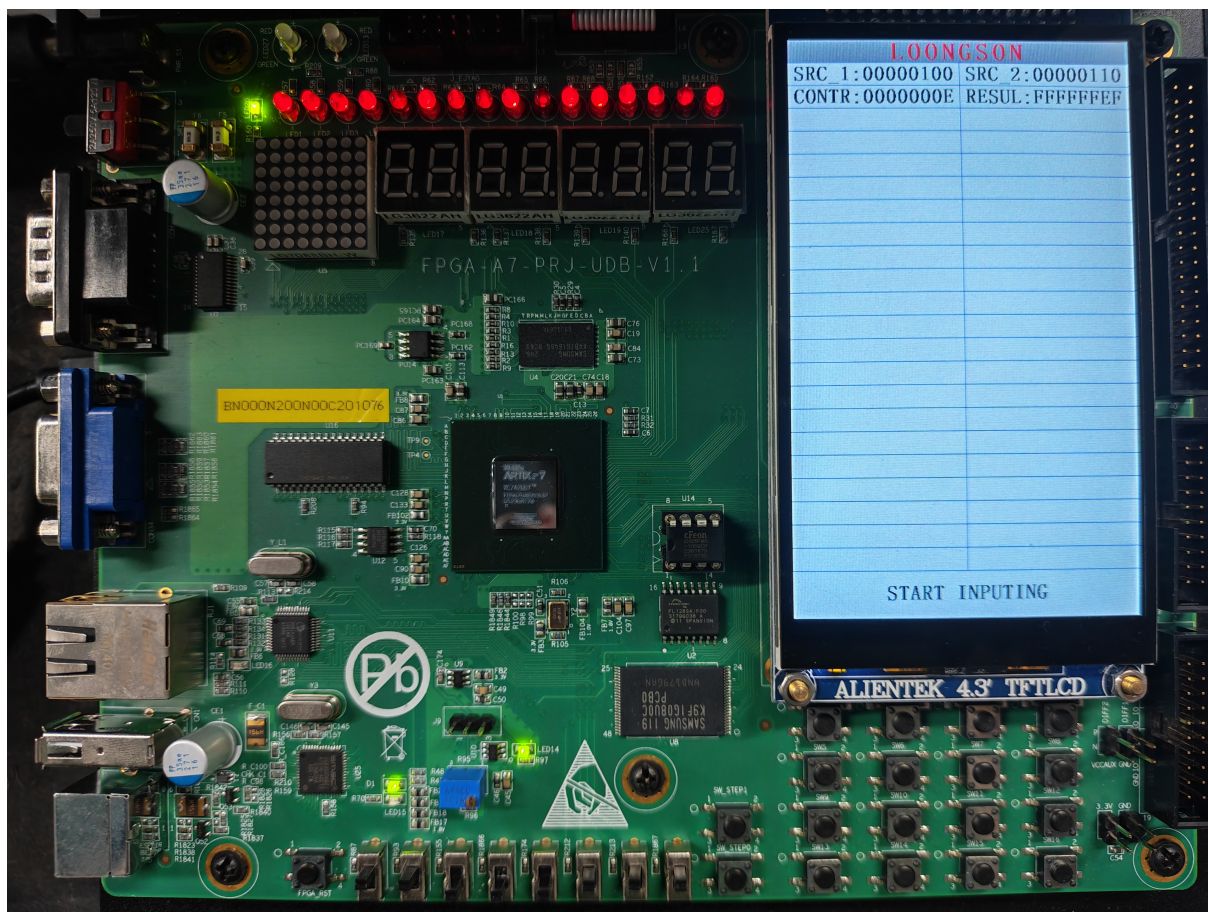


图 3: 按位同或功能

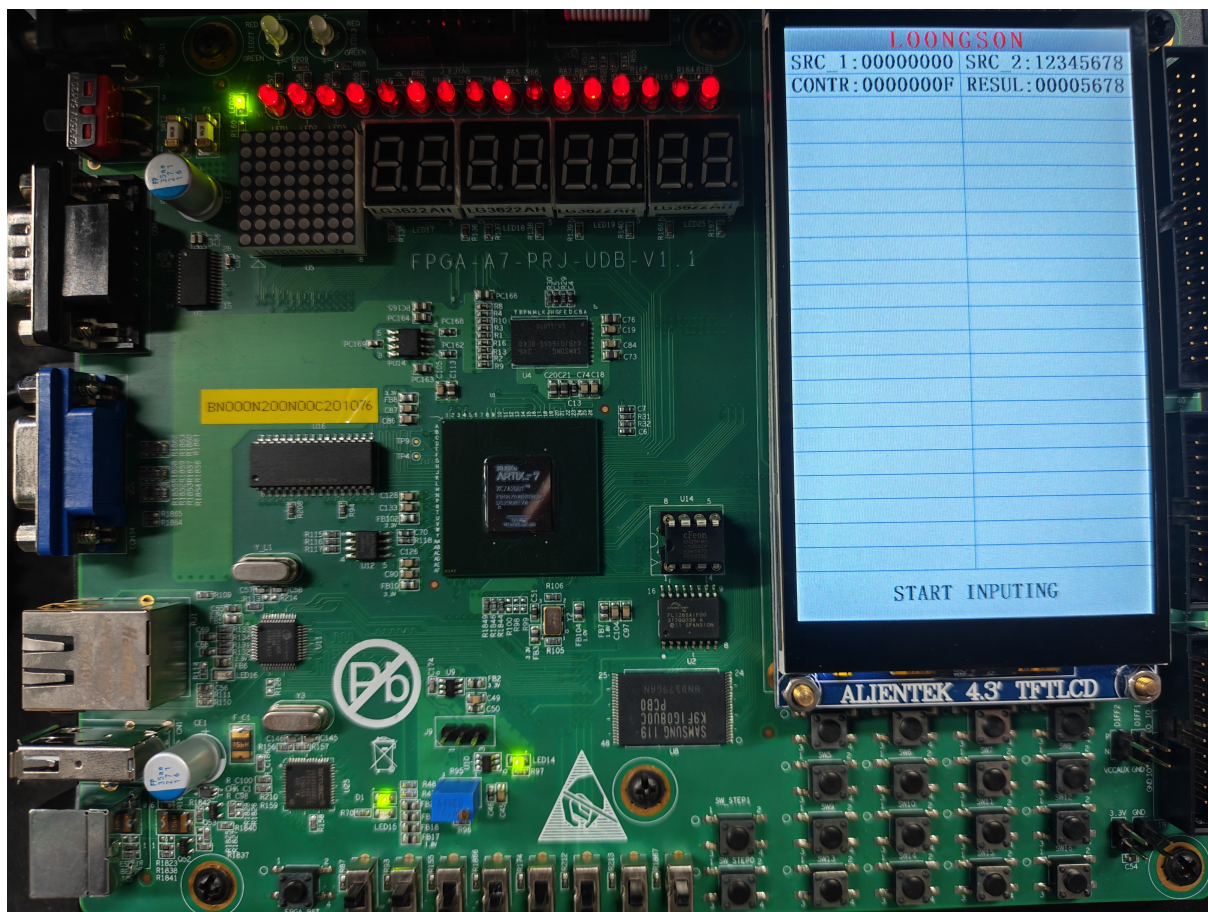


图 4: 低位加载功能